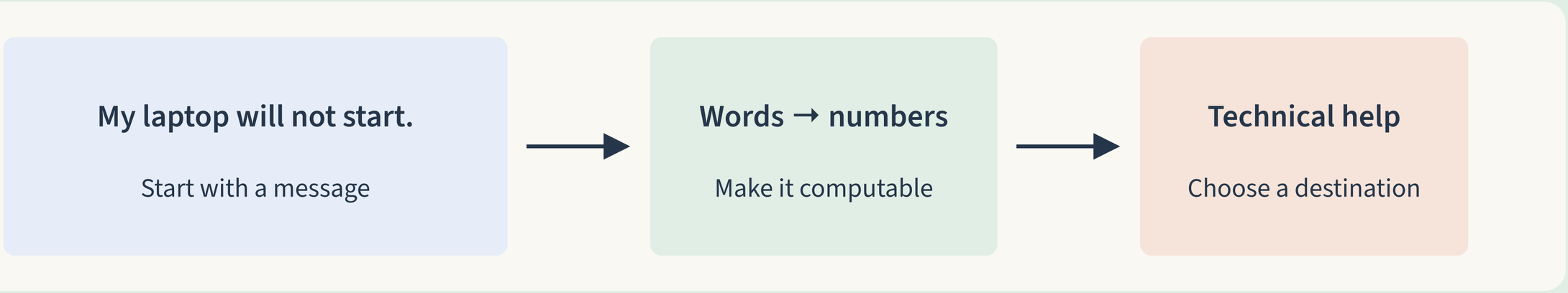
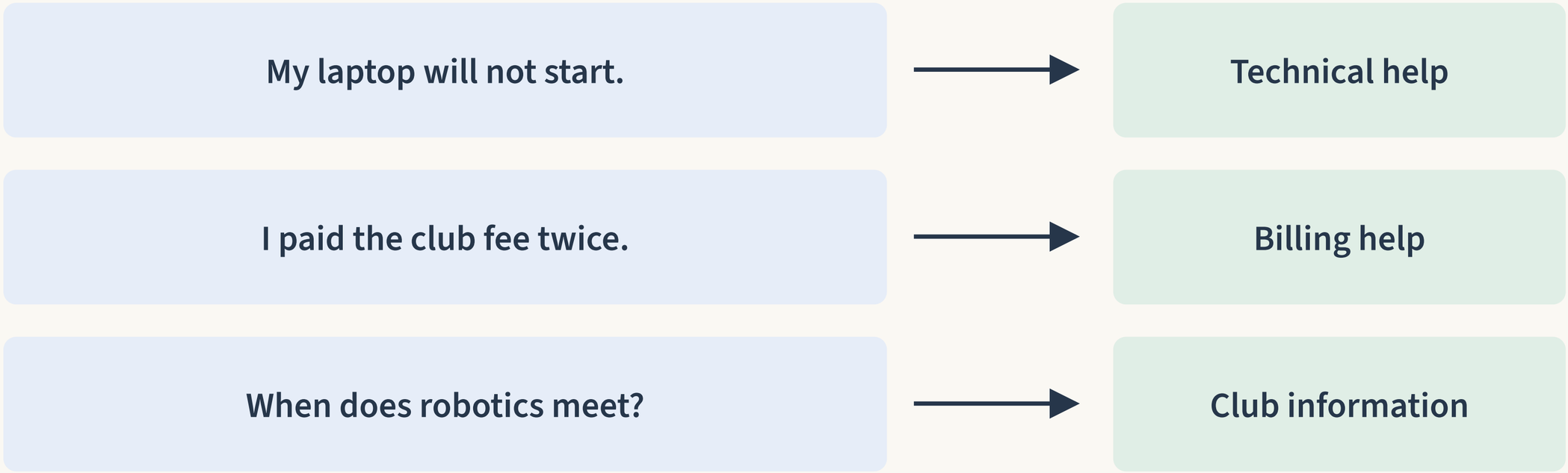
From Bag-of-Words to Jev

How does a computer turn a message into a useful decision? Follow one help-desk example from simple word counts to contextual models and Jev.



An inbox needs a destination

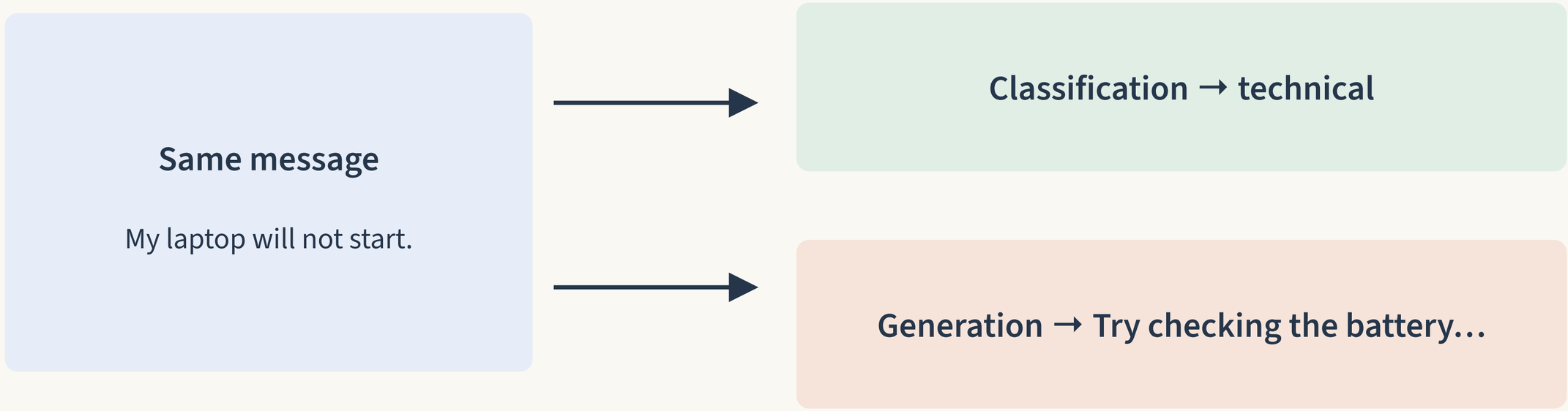
A school help desk receives different requests. Classification assigns each message to a category the software can use.



- A category is also called a class; its name is a label.

Choosing a label and writing a reply

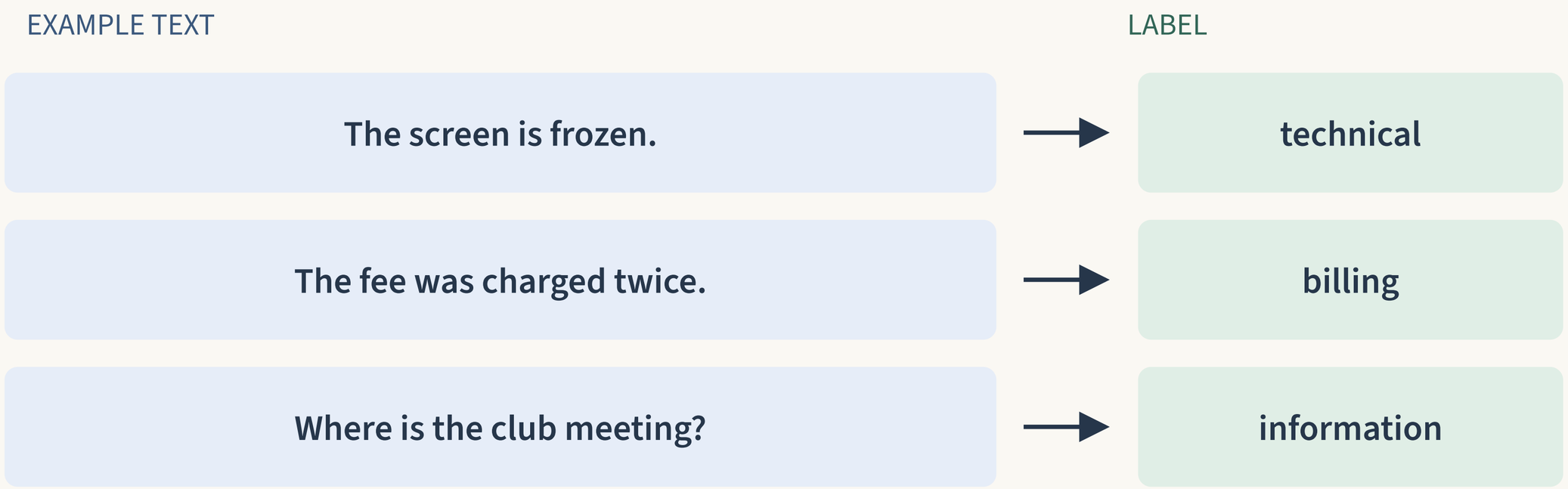
A classifier can choose “technical.” A generator can write troubleshooting advice. An application may need both capabilities.



- Define the output you need before choosing a model.

Examples provide an answer key

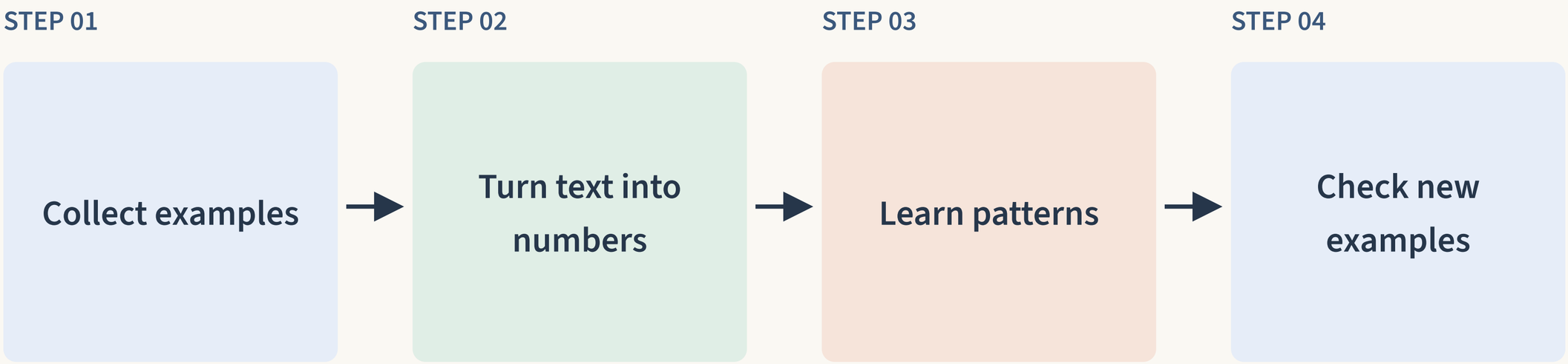
In supervised learning, people supply example texts and their labels. The model learns patterns that help predict labels for new texts.



- A training example pairs an input with a known answer.

The same four steps keep returning

We collect examples, represent the text, learn a decision rule, and check it on new examples. The representation changes across generations of models.

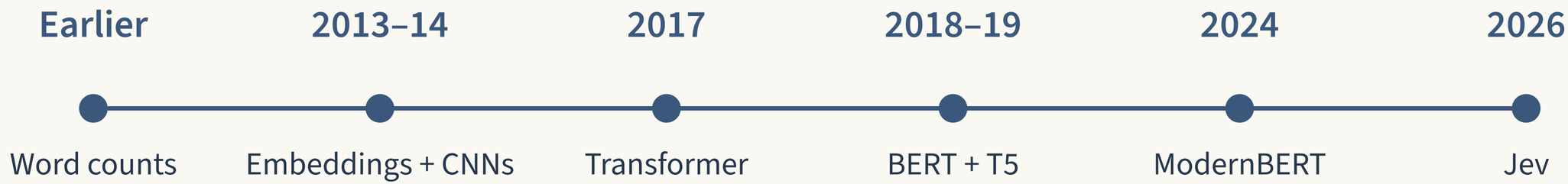


- A representation is a form of the input that the model can compute with.

A timeline of useful ideas

These milestones changed how models represent language and return answers. Earlier methods remain useful when their strengths match the task.

SELECTED MILESTONES · SPACING IS NOT A TIME SCALE



RNNs began earlier; ULMFiT demonstrated language-model transfer for classification in 2018.

- A newer architecture does not guarantee a better result on your dataset.

First, decide what counts as a token

For our first model, we lowercase the text and split it into words. Real tokenizers may also split words into smaller pieces.

STEP 01

Screen, screen frozen!

Original text

STEP 02

screen | screen | frozen

One simple tokenizer: lowercase and split words

- A token is one unit produced by the tokenizer.

Give every known word a position

A vocabulary is the list of words found in the training examples. Each word gets a fixed position in a list of numbers.

fee	frozen	meeting	screen	twice
Position 0	Position 1	Position 2	Position 3	Position 4

Keep these positions fixed when a new message arrives.

- Fit the vocabulary using training examples only.

Count into those positions

The message “screen screen frozen” becomes a vector. Most positions are zero because the message uses only a few vocabulary words.

MESSAGE: “screen screen frozen”

POSITION 0	POSITION 1	POSITION 2	POSITION 3	POSITION 4
0	1	0	2	0
fee	frozen	meeting	screen	twice

The vector is [0, 1, 0, 2, 0]. Its length stays fixed when messages get longer.

- Bag-of-words represents a whole document by its word counts.

Naive Bayes combines word evidence

Begin with how common each class is, then multiply by the word evidence within that class. These invented numbers show how “screen frozen” favors technical help.

MESSAGE: “screen frozen”

CLASS	STARTING SHARE		WORD CLUES		RESULT
technical	**0.60**	×	**0.20 × 0.15**	=	**0.018**
billing	**0.40**	×	**0.02 × 0.01**	=	**0.00008**

Technical has the larger score. Normalize the scores if you need probabilities.

- The result is a comparison score; divide by the total to obtain probabilities.

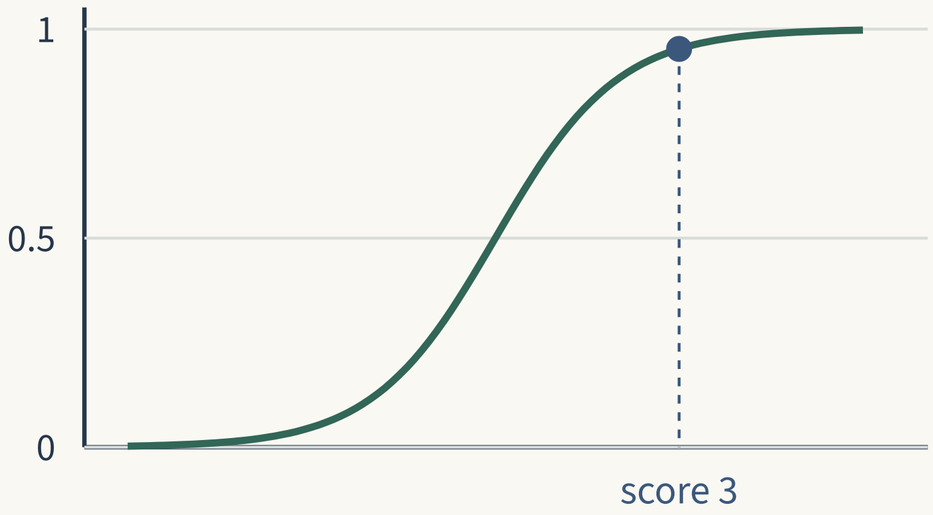
Another option is weighted evidence

For a two-class decision, logistic regression adds weighted word counts and a starting offset. A smooth curve converts that evidence score into a probability.

ADD THE LEARNED CONTRIBUTIONS

frozen: 1 × (+2)	**+2**
screen: 2 × (+1)	**+2**
fee: 0 × (−2)	**0**
starting offset (bias)	**−1**
Evidence score	**3**

CONVERT TO A PROBABILITY



Probability ≈ 0.95 in this example

- Training learns the weights. The numbers here illustrate the calculation.

Some words are more informative

A word appearing in every training document tells us little about which document is different. TF-IDF combines its frequency here with its rarity across documents.



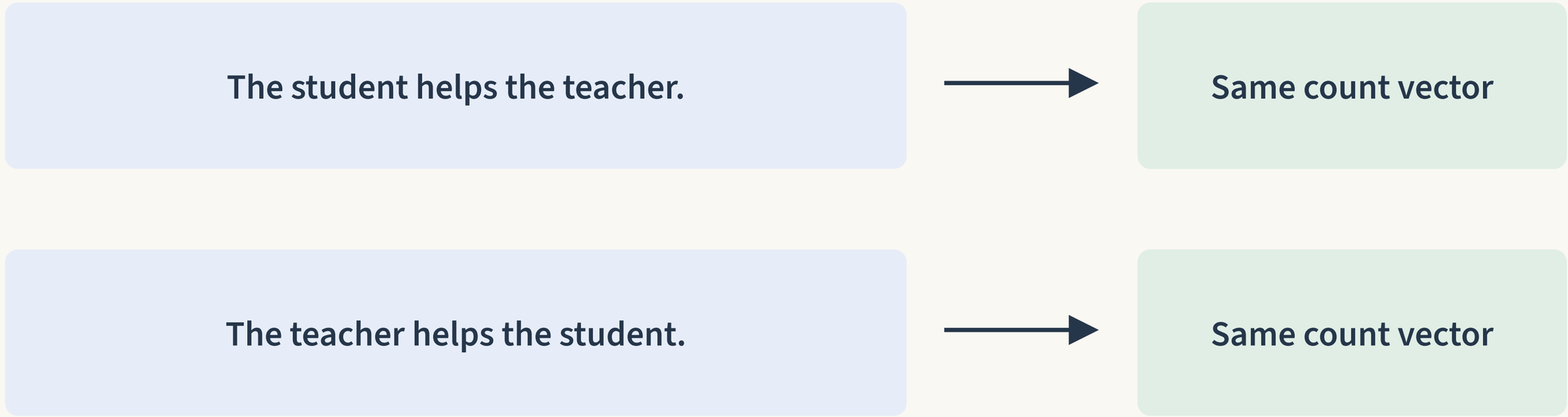
Present in 6 of 6 → less distinctive Present in 2 of 6 → more distinctive

TF counts within one document; IDF uses document frequency across training data.

- Rarity can make a useful clue stand out, but it does not guarantee relevance.

Counting loses the sentence order

These sentences describe different situations, yet they contain the same words with the same counts. A count-only classifier cannot distinguish them.



- If the representation loses a distinction, the next model cannot recover it.

Word pairs restore a little order

A bigram is a pair of neighboring tokens. Adding these pairs can help distinguish phrases such as “not helpful” from “very helpful.”

student helps

A two-word feature

helps the

A two-word feature

the teacher

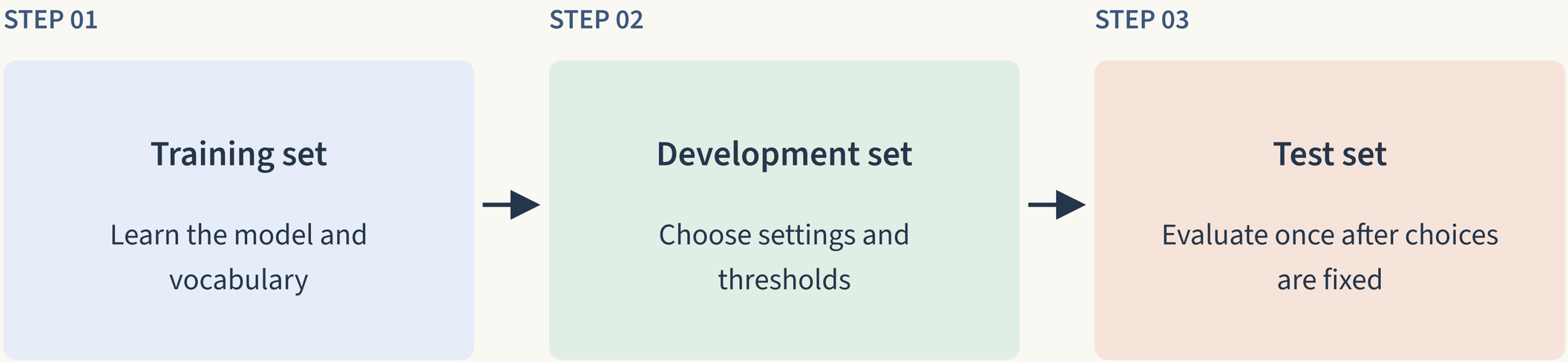
A two-word feature

Pairs preserve local order; they still do not capture every long-distance relationship.

- Useful features can improve a simple model without replacing it.

Keep an honest test set

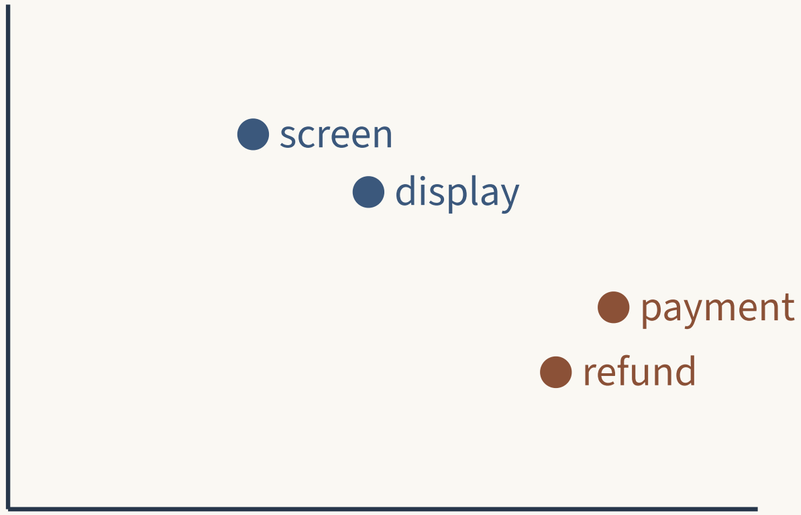
Learn from training data, choose settings on development data, and evaluate the final choices on test data that stayed separate.



- A test set loses its purpose if you repeatedly tune to its answers.

Embeddings replace isolated counts

An embedding represents a word or token with a learned vector. Words used in similar contexts can acquire similar representations.



screen → [0.2, −0.7, 0.5, …]

Values are learned; axes are usually not named concepts.

This invented map illustrates similarity; it is not a measured embedding plot.

- A two-dimensional drawing simplifies a vector with many coordinates.

Context changes what a word means

A static embedding gives “bank” one lookup vector. A contextual model can build different representations from the words around it.

We sat by the river bank.

Nearby words suggest a river edge.

We opened a bank account.

Nearby words suggest a financial institution.

Static lookup: one bank vector

Contextual model: a representation shaped by the sentence

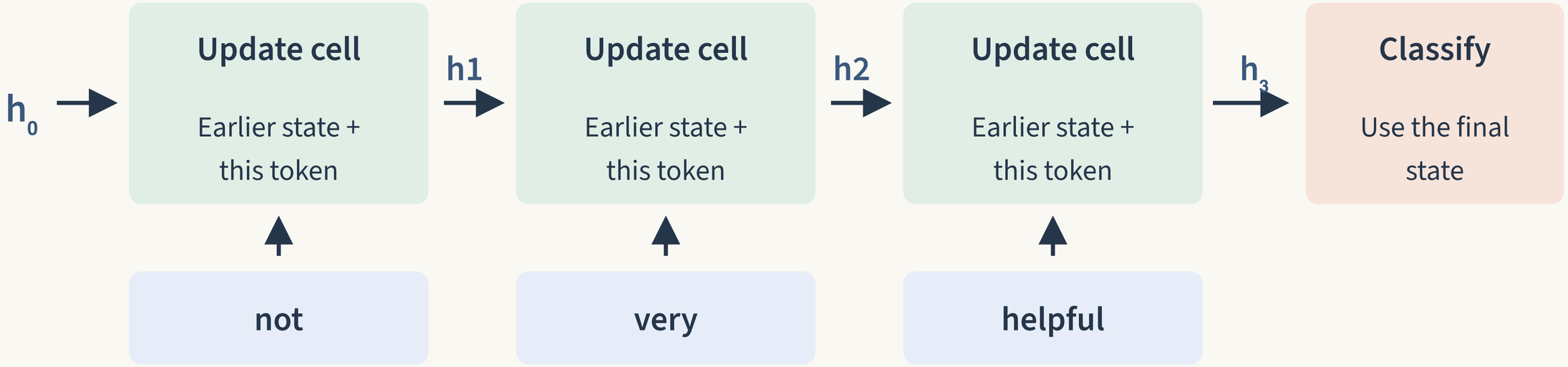
- Meaning depends on the sentence, not only the word.

BERT paper

An RNN carries a running state

A recurrent neural network, or RNN, reads one token at a time. Each update combines that token with a numerical memory of the earlier tokens.

ONE UPDATE RULE, REUSED AT EACH POSITION

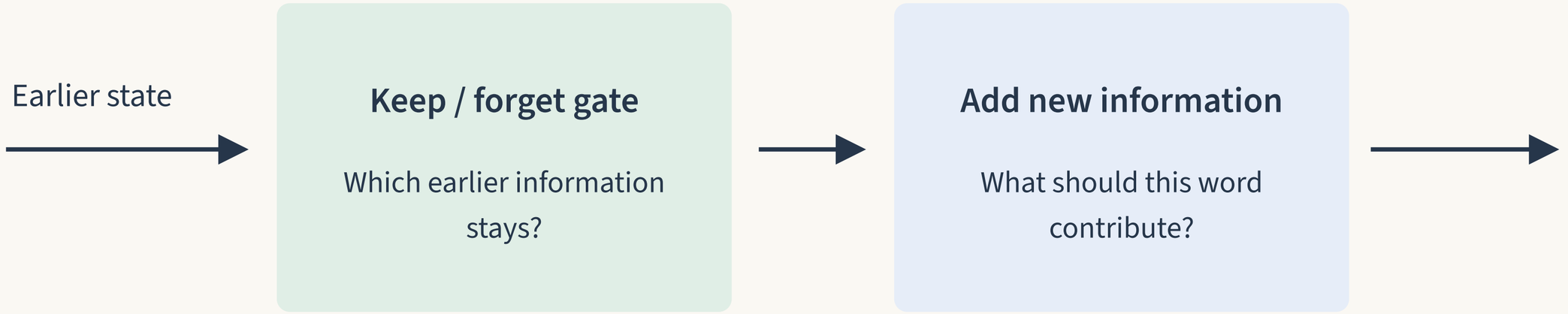


h is a numerical memory of what the model has read so far.

- The boxes show repeated uses of the same learned update rule.

Gates help retain useful information

An LSTM is an RNN with learned gates: numerical controls that keep some old information and add new information. This helps useful clues survive across many steps.

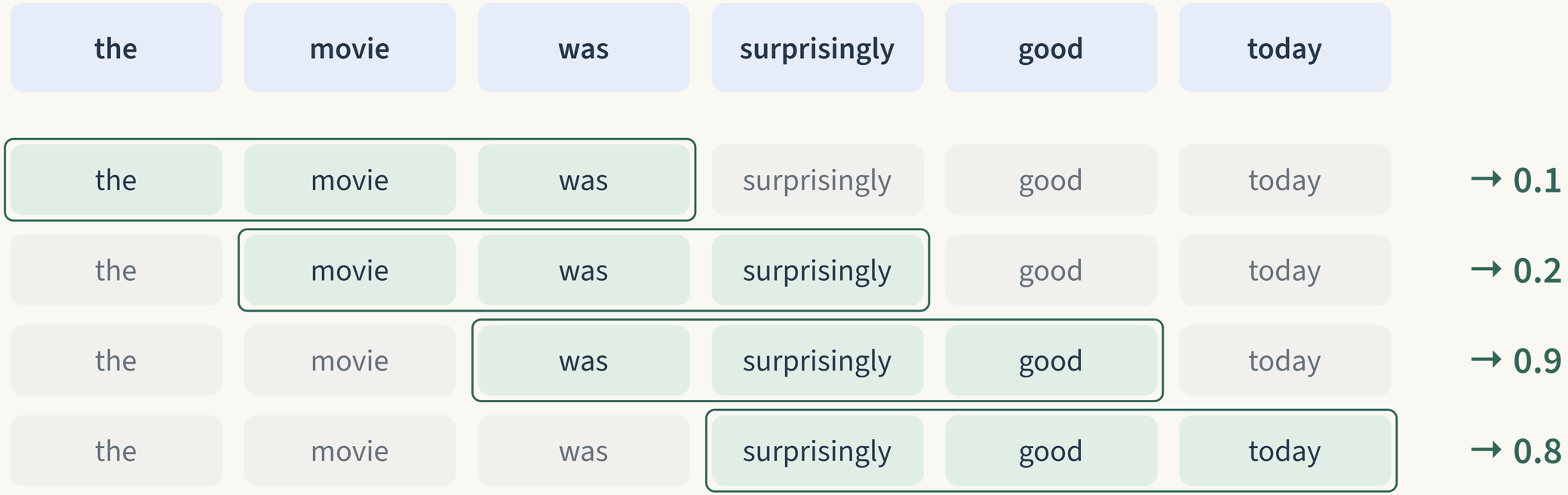


Gates are learned numerical operations, not hand-written language rules.

- LSTM means long short-term memory; it is a kind of RNN.

A CNN looks for local patterns

A convolutional neural network, or CNN, slides a learned pattern checker across small token windows. The same filter produces a response at each position.

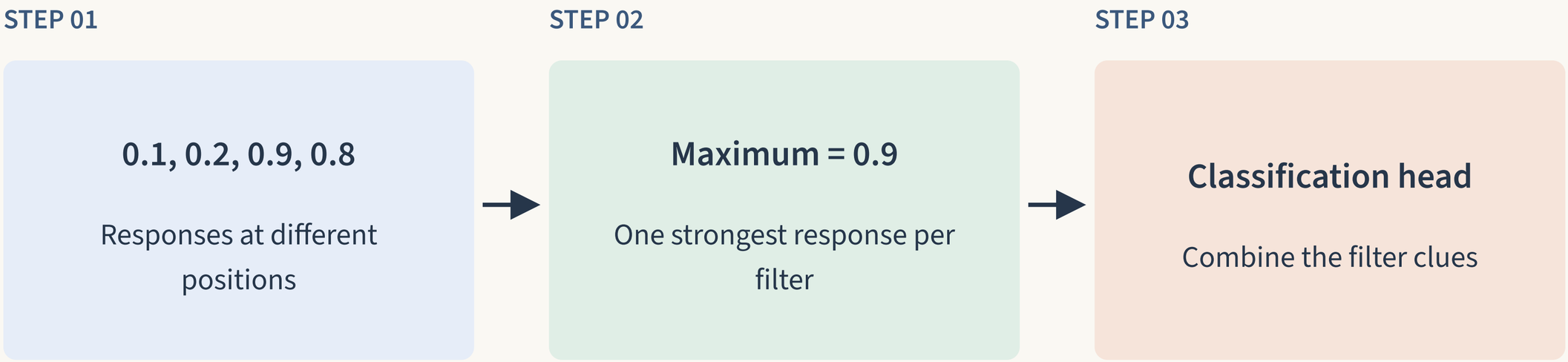


Outlined triplet = one window. Every window uses the same learned filter.

- The responses are invented. The shared filter can learn useful local phrases.

Pooling keeps a compact summary

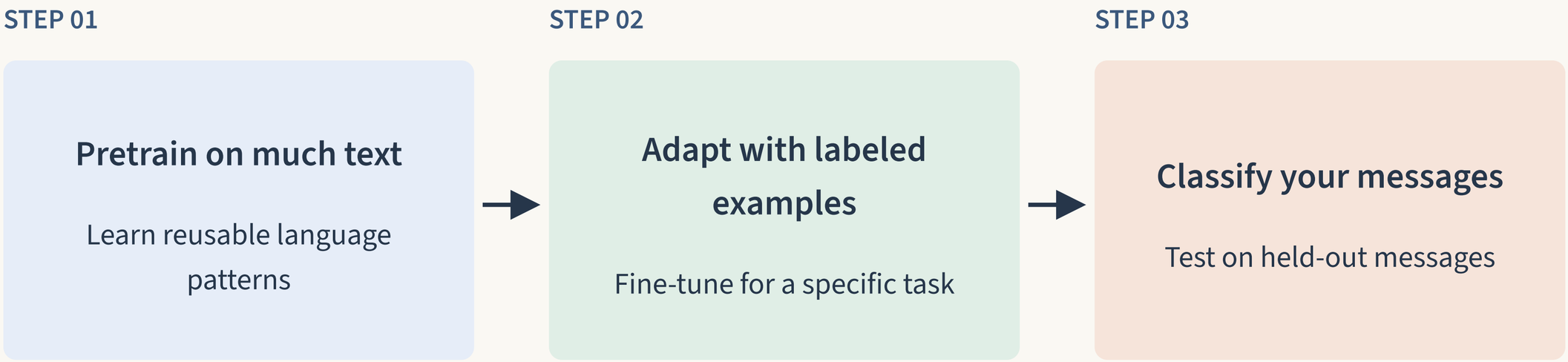
A filter produces several responses, even when messages have different lengths. Max pooling keeps its strongest response; other filters contribute their own summaries.



- A classification head is the final small layer that converts these clues into label scores.

Learning can transfer between tasks

Pretraining learns reusable language patterns from much text. Fine-tuning then adapts the model to a smaller labeled classification dataset.

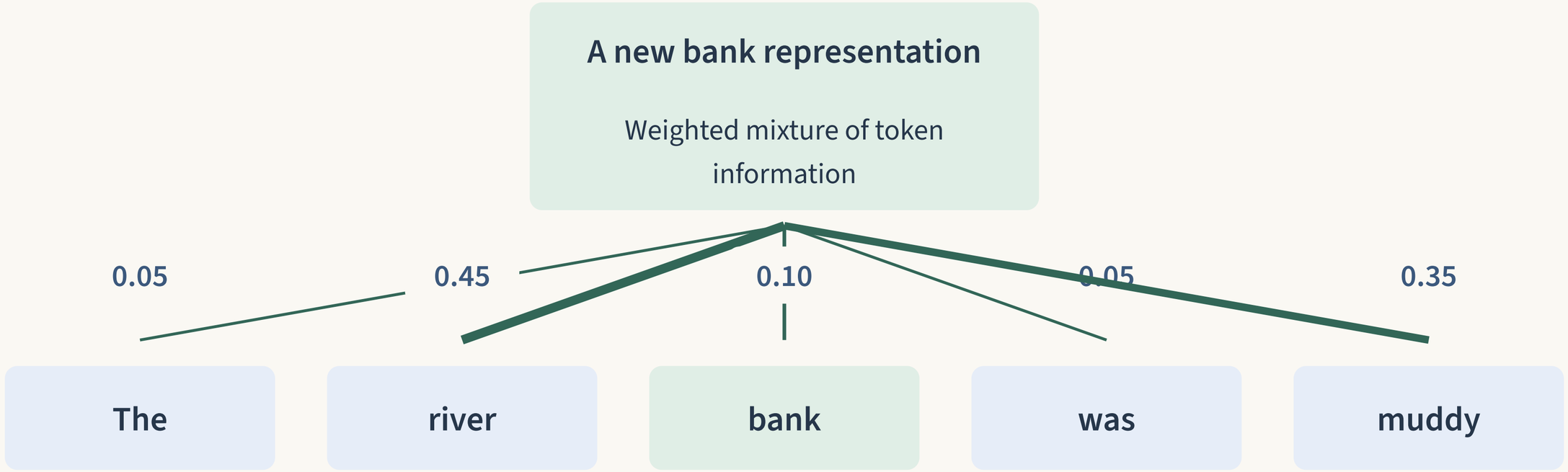


- ULMFiT showed this approach with an RNN in 2018.

ULMFiT paper

Attention connects relevant tokens

To update “bank,” attention combines information from token representations using learned weights. Context such as “river” and “muddy” can help shape the result.



Invented weights sum to 1. Position information also helps distinguish word order.

- These weights illustrate mixing; they are not measurements or a proof of reasoning.

Two useful ways to read context

Read a row to see which positions a token may use. An encoder can read both sides; a causal decoder can read the current position and earlier ones.

ENCODER: BOTH DIRECTIONS

Read information from →

	The	river	bank	was	muddy
The	✓	✓	✓	✓	✓
river	✓	✓	✓	✓	✓
bank	✓	✓	✓	✓	✓
was	✓	✓	✓	✓	✓
muddy	✓	✓	✓	✓	✓

CAUSAL DECODER: CURRENT + EARLIER

Read information from →

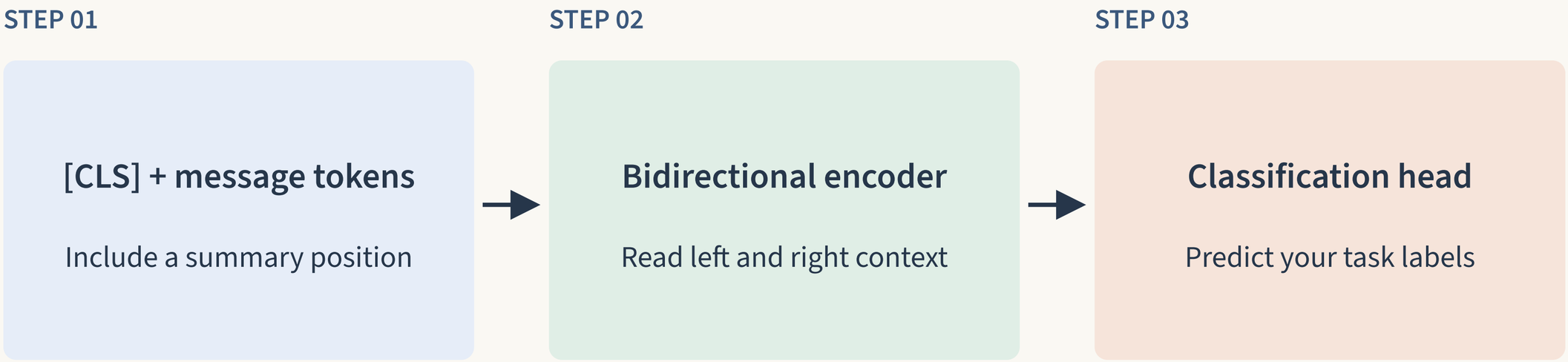
	The	river	bank	was	muddy
The	✓	×	×	×	×
river	✓	✓	×	×	×
bank	✓	✓	✓	×	×
was	✓	✓	✓	✓	×
muddy	✓	✓	✓	✓	✓

Rows = token being updated · columns = source tokens · ✓ allowed · × blocked

- For “bank,” the causal mask blocks “was” and “muddy,” which come later.

BERT reads before classifying

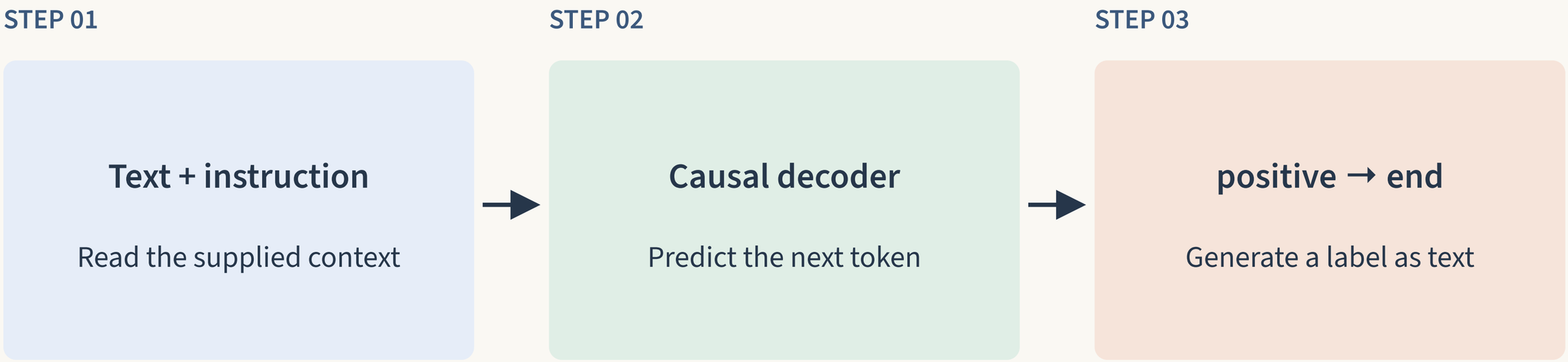
An encoder builds representations using context on both sides. In a BERT-style classifier, a summary position called [CLS] feeds a small layer trained to predict your labels.



- ModernBERT updates the encoder approach rather than abandoning it.

A decoder can generate a label

A causal decoder predicts the next token from the earlier context. Given “classify this review,” a generative model can write a label such as “positive.”



- A label written as text is still produced through token generation.

A decoder can also use a small head

A causal model can also feed a small trained classifier. Its final input position has access to the full message, so that representation can support a label prediction.

STEP 01

Text tokens

Use a causal model

→

STEP 02

Final non-padding token

This position can see the full input

→

STEP 03

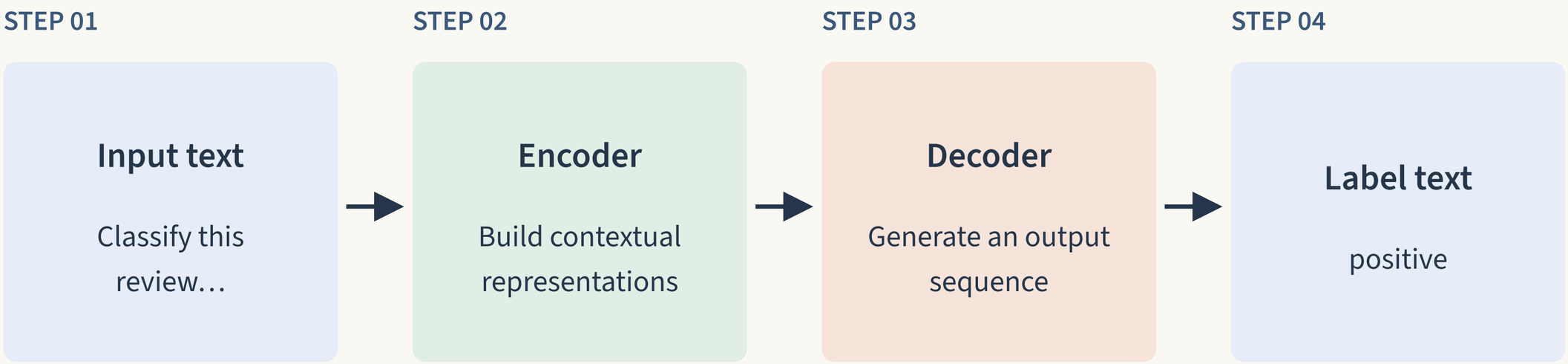
Small classification head

Predict labels directly after task training

- A classification head offers another route besides prompting for label text.

T5 treats tasks as text in, text out

An encoder reads the input, and a decoder writes the answer. A classification answer can be a short label expressed as text.



- T5 is an encoder–decoder model, introduced in 2019.

Choose the approach for the job

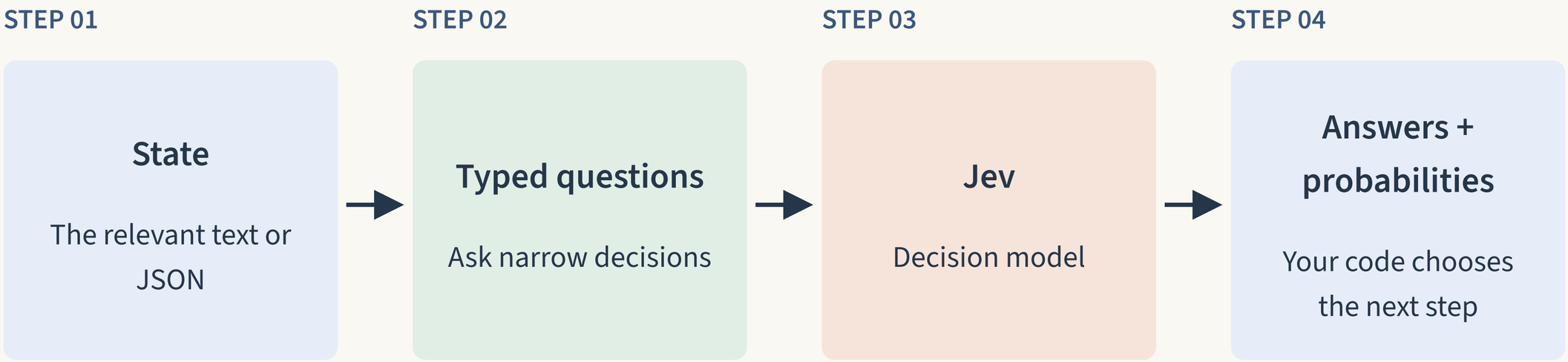
A small classifier, a fine-tuned encoder, a generative model, and a hosted decision model solve overlapping tasks with different practical tradeoffs.

APPROACH	USEFUL OUTPUT	TASK SETUP
Word-count classifier	One fixed label set	Train on labeled examples
Fine-tuned encoder	Contextual classification	Adapt to your label set
Generative model	Labels and open text	Prompt; sometimes fine-tune
Jev decision API	Typed bounded decisions	Describe; test on your task

- Compare quality on your data along with setup effort, latency, and cost.

System One is a decision interface

TypeSafe calls this a System One interface: give a model the relevant evidence and a few narrow questions, then receive typed decisions. Jev is its flagship model.



- The model predicts an answer. Your code decides what happens next.

Separate the interface from the internals

We can explain Jev’s public inputs, outputs, and stated training goal. The full architecture and training procedure have not been disclosed.

Publicly documented

State + questions; typed answers. TypeSafe names its training approach RLCD.

Not fully disclosed

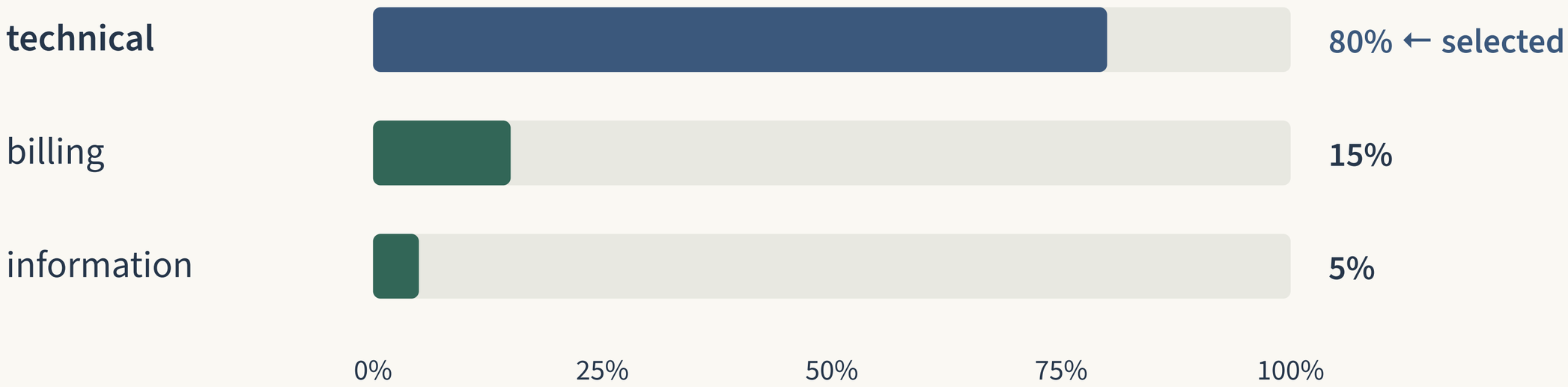
Architecture, training recipe, reward design, and full training data.

- A useful interface description is different from a verified diagram of the model’s internals.

Choice selects one described option

Give each option a clear description. The answer includes the selected option, a probability for every option, and a confidence value.

ONE QUESTION · ONE SELECTED OPTION

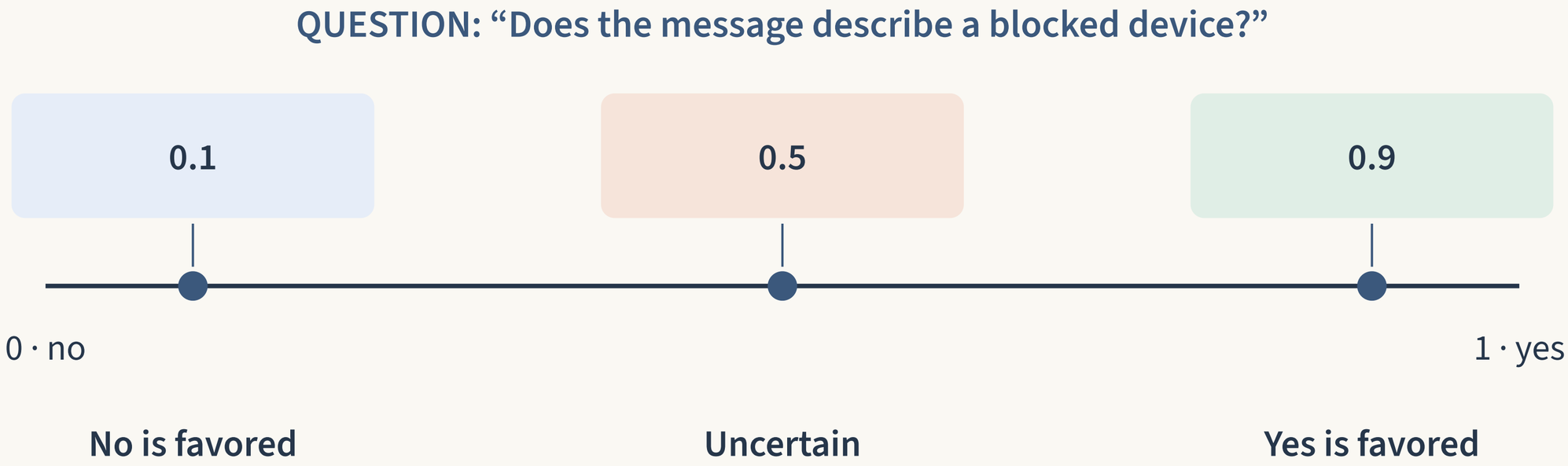


Illustrative probabilities total 100%. The selected label is technical.

- These three probabilities are invented and sum to 1.

Noul asks a yes-or-no question

Noul estimates whether one statement is true. A value near 1 favors yes, near 0 favors no, and near 0.5 leaves the answer uncertain.

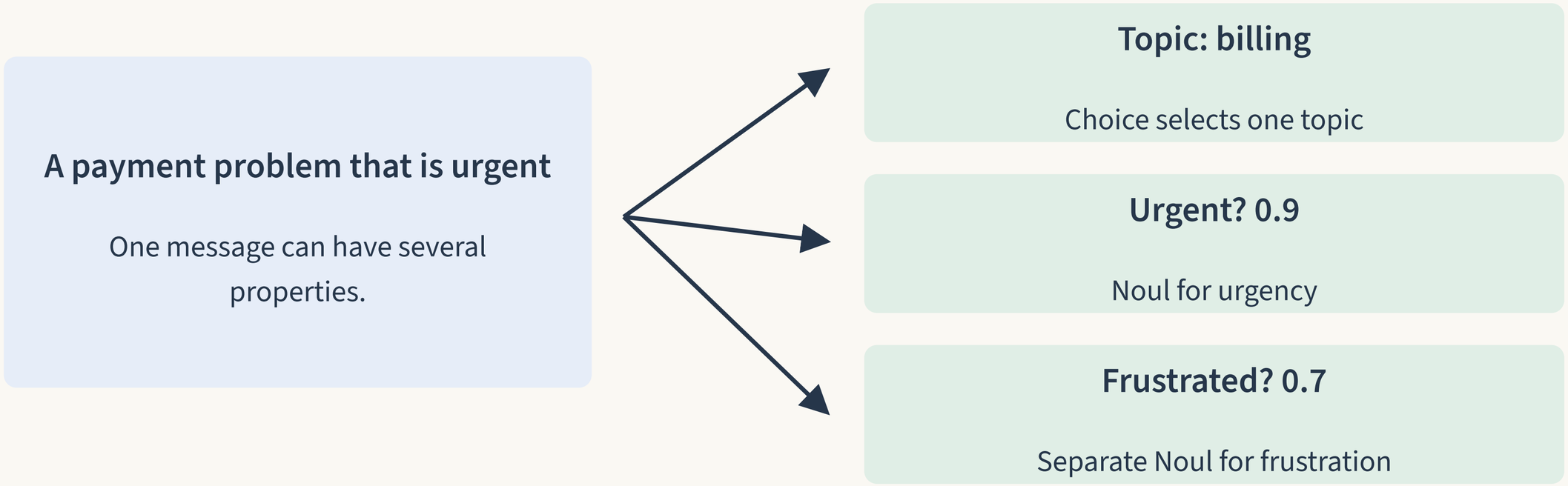


For this question, P(no) = 1 − P(yes). Separate questions need not sum to 1.

- Noul is P(yes) for one question; it does not include a separate confidence field.

Several properties can be true together

Use one Choice for a single topic, and separate Noul questions for properties such as urgency and frustration. Their probabilities do not need to sum across questions.



- Single-label categories and multi-label properties answer different questions.

Score places an input on a rubric

Score uses ordered levels that you describe in a rubric. It averages their positions using the predicted probabilities, so the returned value can fall between levels.

Level 0: 10%

Cosmetic issue

Level 1: 30%

Workaround exists

Level 2: 60%

Blocked; no workaround

$$\text{Score} = 0 \times 0.1 + 1 \times 0.3 + 2 \times 0.6 = 1.5$$

A position on your rubric, not a physical measurement of damage.

- A score of 1.5 is an expected rubric position, rather than a new category.

Confidence differs from probability

For three Choice options, an even split gives each one ⅓. Jev's confidence rescales the top probability relative to that baseline; it is a different number from P(top option).

SAME ANSWER, TWO DIFFERENT NUMBERS

Top probability: 0.80

Estimated probability of the selected option

Choice confidence: 0.70

Distance above an even split, rescaled to 0–1



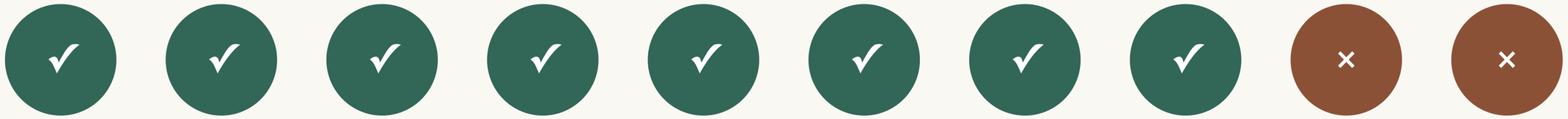
confidence = (0.80 − ⅓) / (1 − ⅓) = 0.70

Compare with observed outcomes before trusting the number as a routing signal.

- Choice confidence depends on how many options you offer and still needs validation.

Calibration is checked over many cases

If events given probability 0.8 occur about 80% of the time on representative data, those estimates are calibrated around that probability.



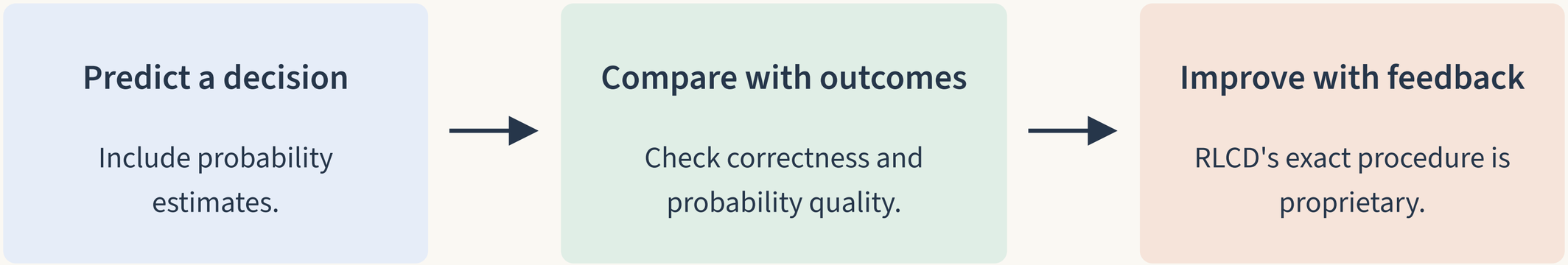
Eight correct would be compatible with calibration in this tiny illustration.

A real check needs many representative labeled examples.

- Accuracy asks which label wins; calibration asks whether the probabilities match outcomes.

RLCD names the training goal

TypeSafe describes training Jev for correct decisions and probabilities that reflect uncertainty. It names the approach RLCD; its exact training procedure remains proprietary.

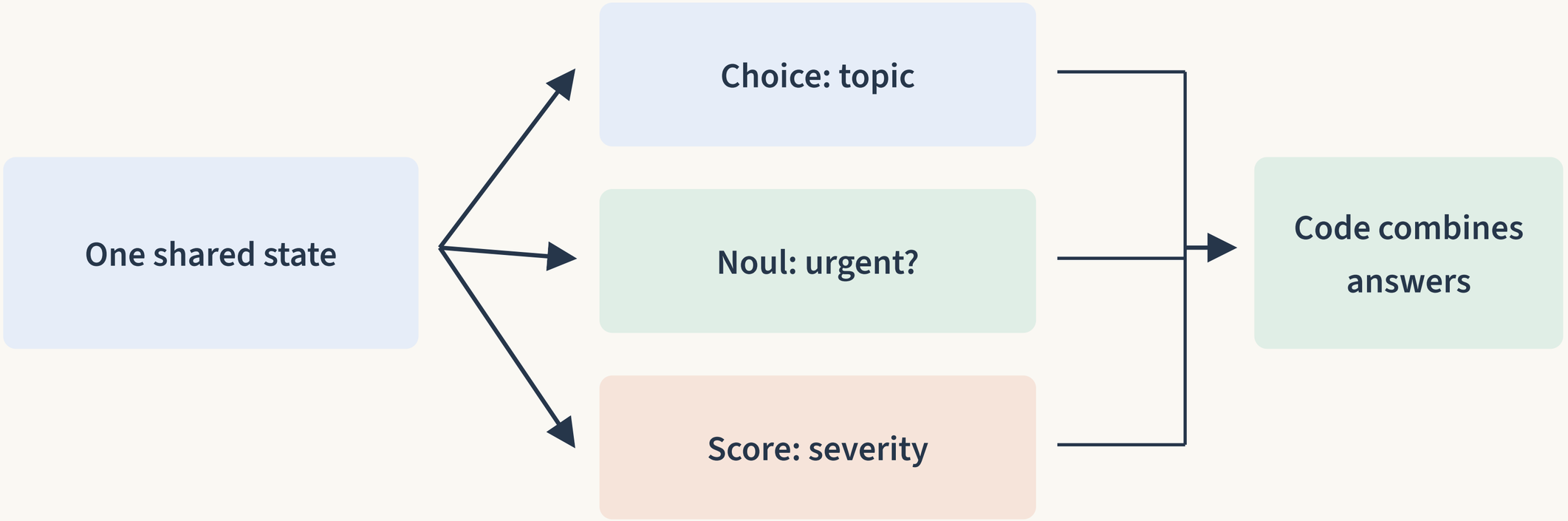


This is a teaching diagram of a training goal, not Jev's internal architecture.

- This diagram explains the goal; it does not reconstruct RLCD.

Ask narrow questions together

Jev evaluates multiple questions about a shared state in one request. Your code can combine the returned answers into a visible policy.



- A question that needs an earlier answer requires a later workflow step.

Supply relevant facts in the state

Text and JSON can provide a message, current policy, and verified facts. Keep instructions in the questions and point them at the fields they should judge.

STATE · RELEVANT EVIDENCE

{

"message": "The device will not start.",

"policy": "Blocked devices → technical",

"facts": {"loan_device_available": true}

}



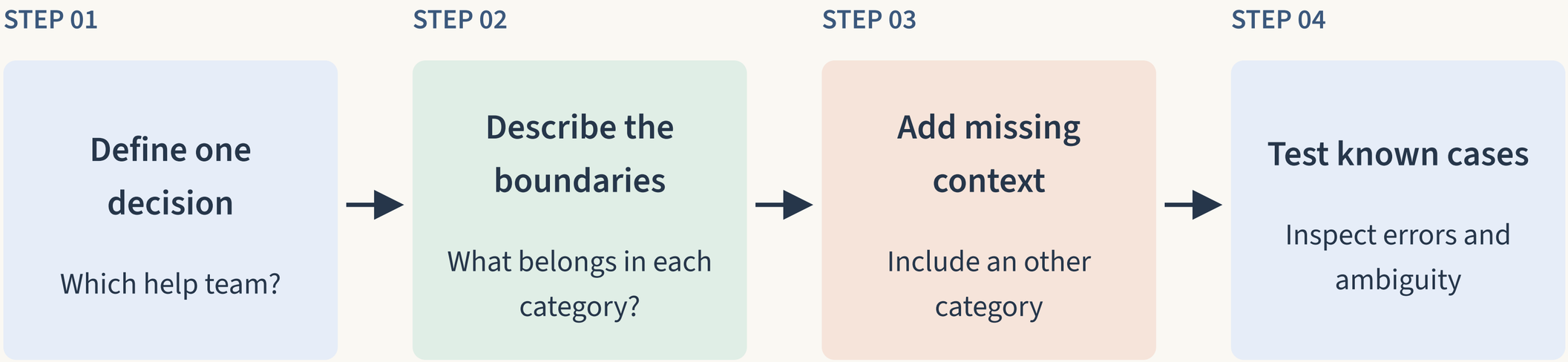
QUESTION

Which team should handle this request?

- Structured inputs help clarify what evidence belongs to each decision.

Write questions with clear boundaries

A useful question defines one decision and gives concrete criteria. Missing context and overlapping categories often need review or clarification.



- Evaluate changes on labeled examples rather than judging them by higher confidence alone.

A request becomes ordinary data

Send model, state, and questions to the API. Read the typed answers, then let explicit code choose the handler.

REQUEST

model, state, questions: instructions + criteria

→

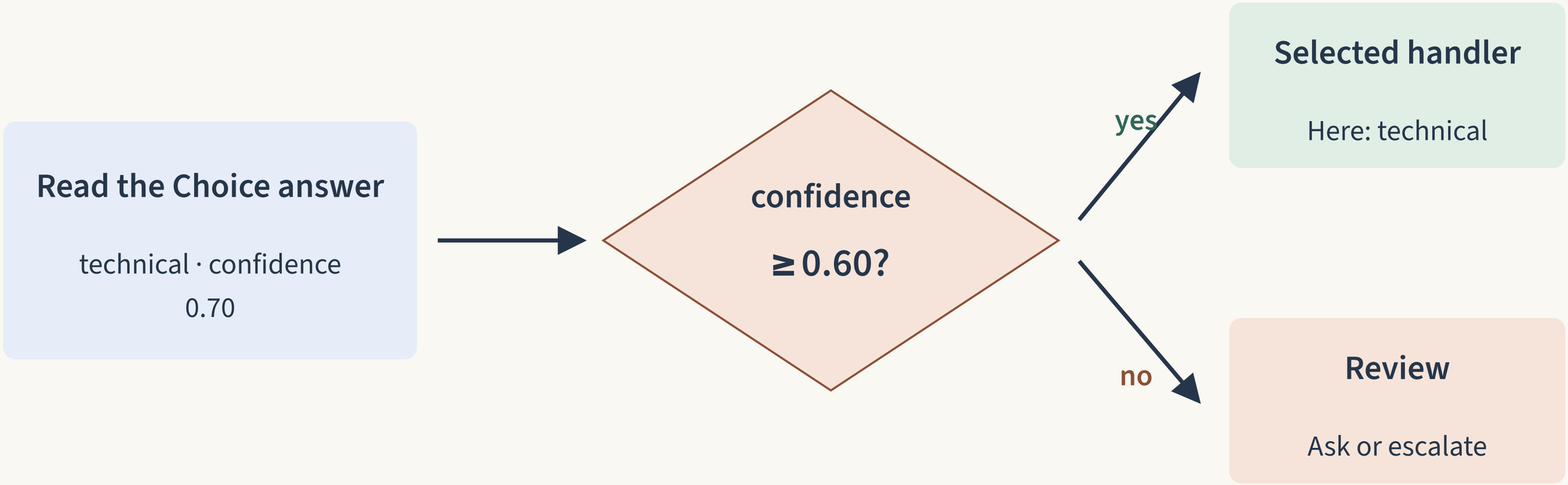
RESPONSE

model, answers: choice / noul / score; usage

- The notebook includes HTTP, Python SDK, and LangChain examples.

Give uncertainty a route

Application code compares Choice confidence with a chosen threshold. Here 0.70 clears an illustrative 0.60 gate, so the message reaches its selected handler.



0.60 is an illustrative policy threshold. Choose yours on development data.

- A threshold is a policy setting. Choose it using development-set errors and review capacity.

More review changes the workload

Raising a confidence threshold sends more cases to review and fewer to automatic handlers. Compare those fractions with error rates to decide whether the tradeoff helps.

CONFIDENCE GATE	SAME 10 CASES AT EACH THRESHOLD	
Threshold 0.2	A A A A A A A A R R	8/10
Threshold 0.5	A A A A A A R R R R	6/10
Threshold 0.8	A A A R R R R R R R	3/10

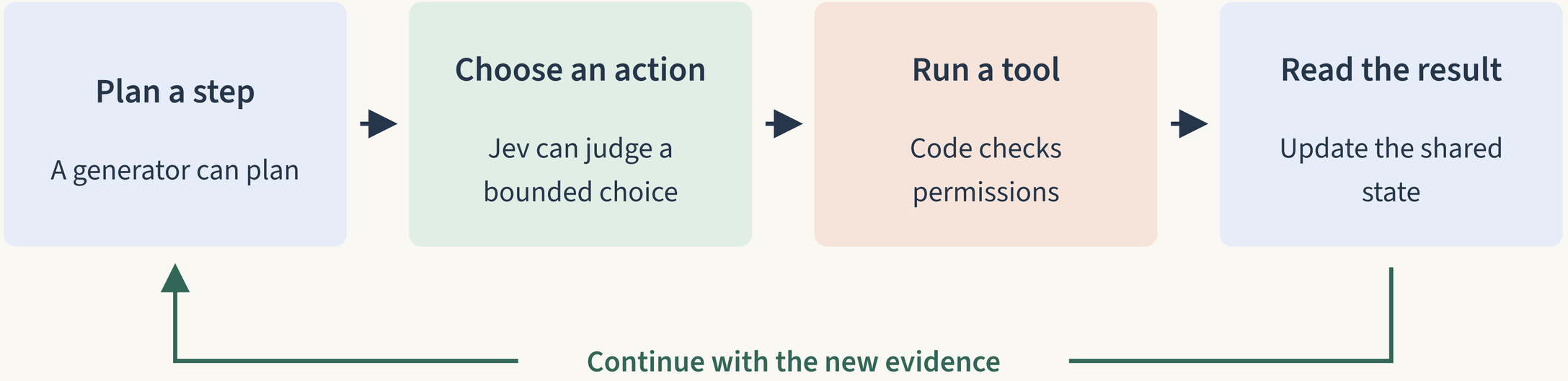
A = automatic route · R = review · Fraction = automatic coverage

Higher thresholds reduce coverage here; measure error rates separately.

- Coverage measures how often you act automatically; it does not measure correctness.

An agent chooses steps toward a goal

An agent takes steps, runs tools, and uses the results to choose what to do next. Jev can judge bounded choices while application code controls the loop and permissions.

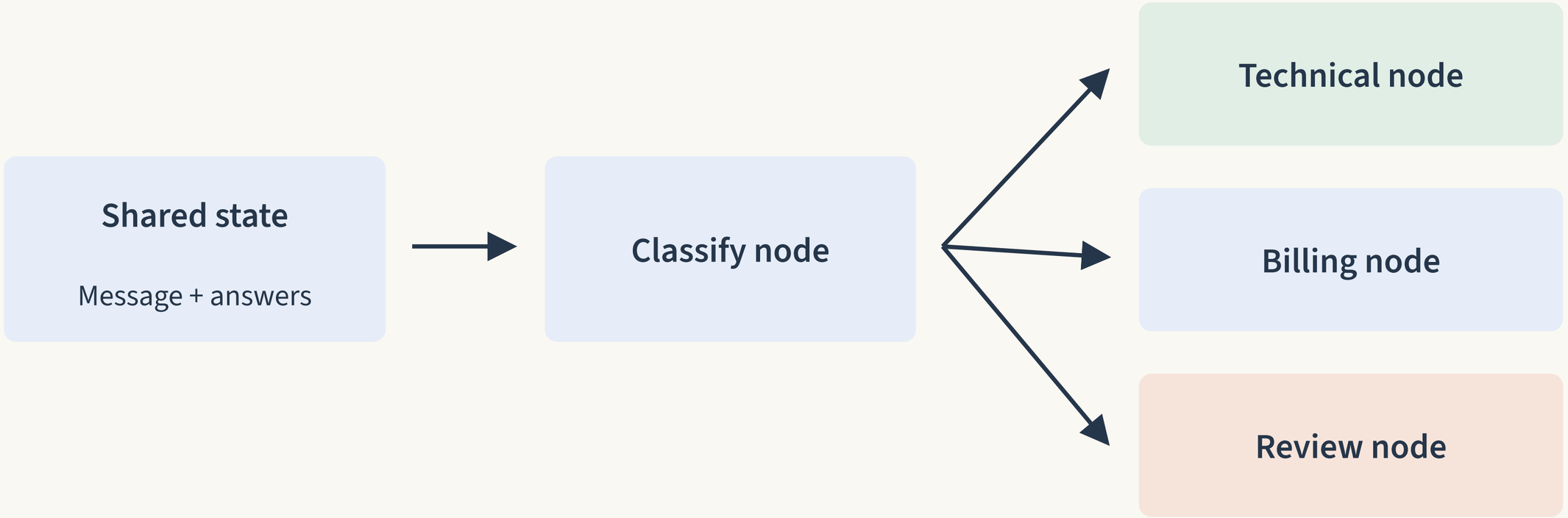


Stop when the task is complete or the workflow reaches a budget limit.

- Tool results become evidence for the next step. Completion and budgets stop the loop.

A graph makes the workflow visible

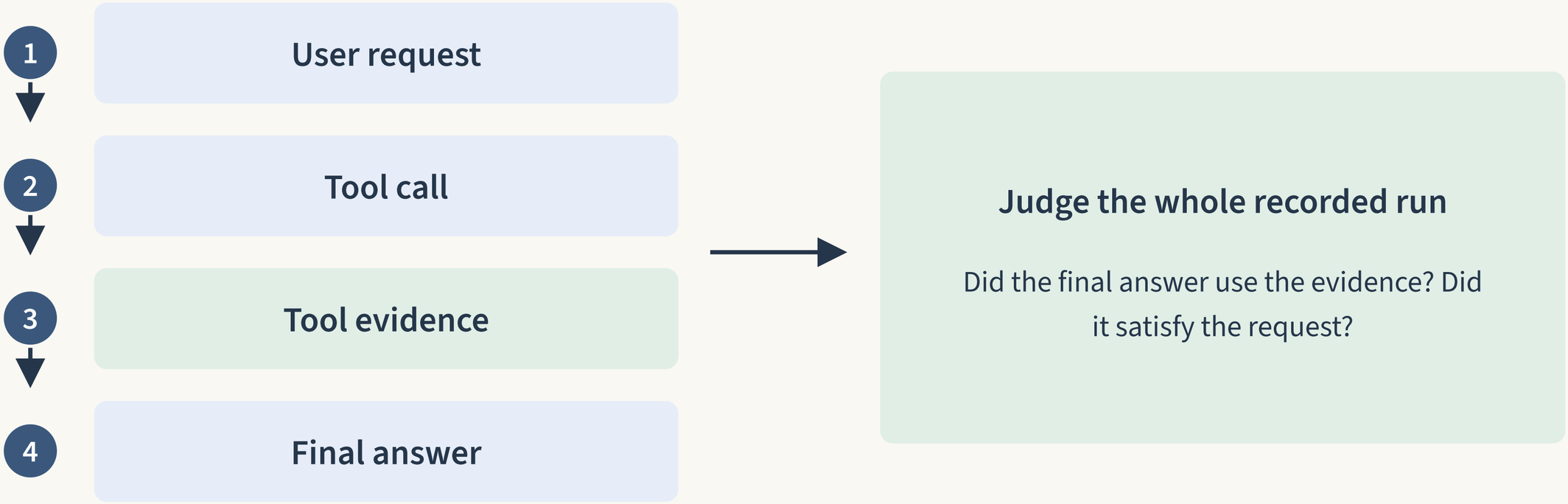
LangGraph represents work as nodes connected by edges. State carries the message and results, and conditional edges select the next node.



- A node may call Jev, run ordinary code, or invoke another model.

Judge the recorded behavior

A trace records what an agent was asked, what it did, and what evidence it received. Focused questions can evaluate whether the final answer used that evidence.



- A judge's score is a model prediction that also needs validation.

Read early evidence at its real scale

LangChain reported strong repeatability and agreement in a small weather-agent judge study. Repeating a fixed run tests stability, not breadth across new tasks.

Fixed run 1	Fixed run 2	Fixed run 3	Fixed run 4	Fixed run 5
100 repeated judgments	100 repeated judgments	100 repeated judgments	100 repeated judgments	100 repeated judgments

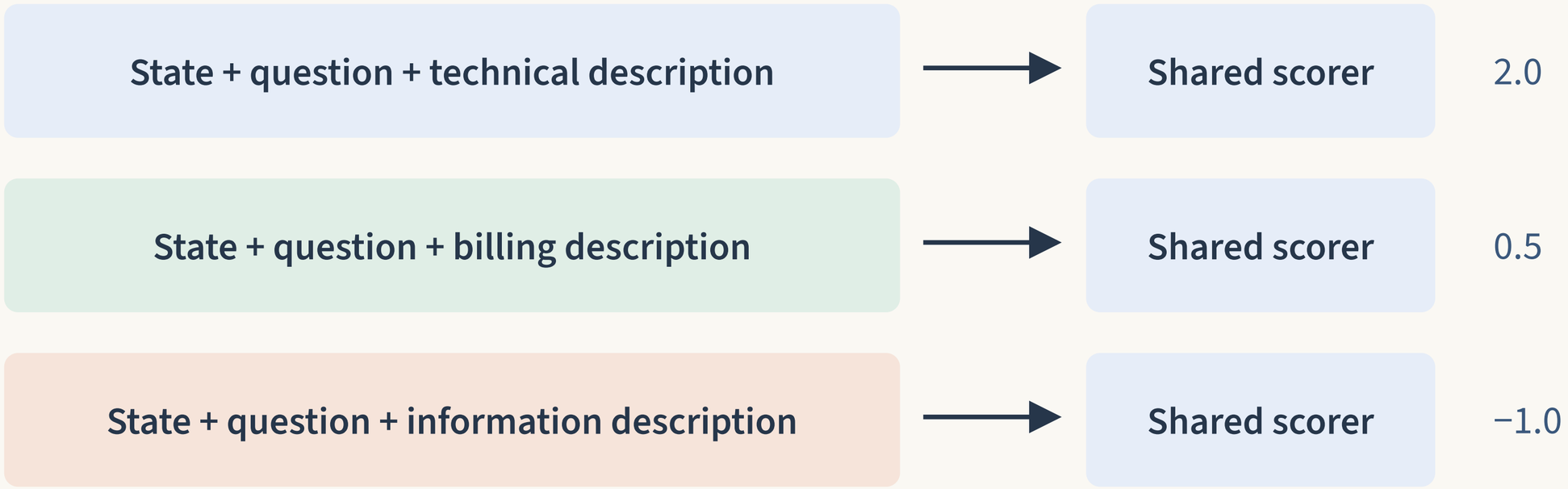
500 judgments, but only 5 distinct agent runs

Repeatability and agreement on new tasks answer different questions.

- The study's observations are useful evidence, not a universal ranking of judges.

A similar API is easier than a similar model

An open-model experiment could score each candidate description with a shared scorer. That illustrates one design, but says nothing definite about Jev's internals.

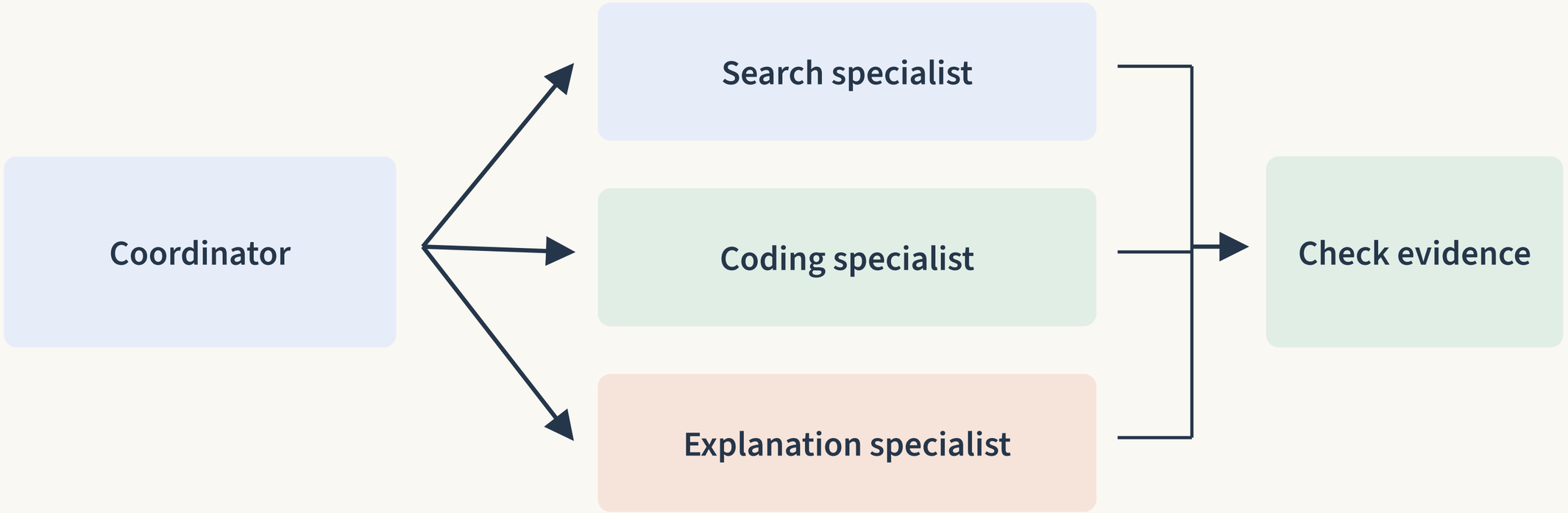


Combine scores into probabilities. Matching the API does not establish matching quality.

- Matching an interface does not demonstrate generalization or calibration.

Could a coordinator route specialist agents?

A decision model might assign work and check handoffs between agents. Research should measure the whole team's success, not only the coordinator's label accuracy.



- Shared mistakes can make several agreeing agents confidently wrong.

Turn an idea into a fair experiment

Choose a hypothesis, separate development from testing, compare alternatives, and inspect the cases where the approach fails.

STEP 01

Ask a testable question

What could improve?

→

STEP 02

Keep a held-out test set

Prevent tuning on the answer key

→

STEP 03

Compare alternatives

Measure quality, latency, cost

→

STEP 04

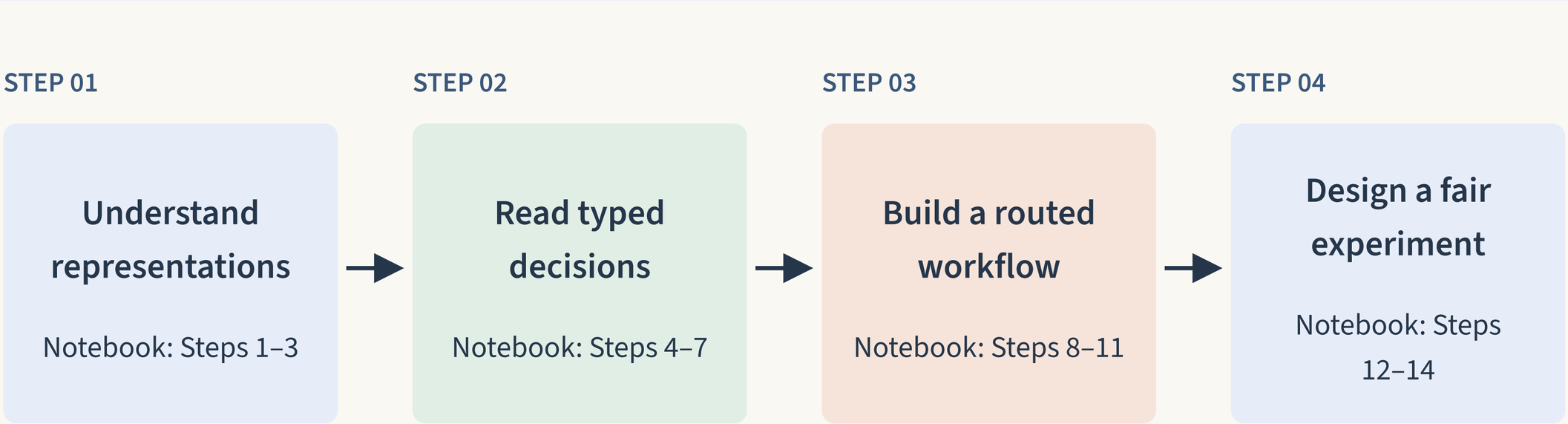
Inspect the failures

Learn where the idea breaks

- The notebook and research roadmap give six concrete projects.

Now build the decision path

The companion notebook starts with a small word-count classifier, then teaches typed decisions, routing, agent evaluation, and research design.



- Open notebooks/system1_tutorial.ipynb and run the offline path first.